09 floyd warshall

Floyd-Warshall Algorithm

1. What is it, and when is it used?

Floyd-Warshall computes the shortest path between **every pair of nodes** in a weighted graph (handles negative edges, but not negative cycles) using dynamic programming, in $O(V^3)$. It's used whenever a problem needs **all-pairs shortest paths**, not just from a single source, especially on graphs that are small-to-moderate in size.

2. Why is it used?

Running Dijkstra or Bellman-Ford from every single node would cost $O(V \cdot (V+E) \log V)$ or $O(V^2 \cdot E)$ respectively — messy and, depending on graph density, sometimes worse than Floyd-Warshall's clean $O(V^3)$. Floyd-Warshall also has an exceptionally simple 3-nested-loop implementation and naturally supports negative edges (just not negative cycles), making it the go-to when V is small (roughly ≤ 400–500) and you need the full distance matrix.

3. Where is it used?

- All-pairs shortest paths in small dense graphs
- Transitive closure of a graph (reachability between all pairs)
- Finding the "city with the smallest number of neighbors within a threshold distance"
- Detecting negative cycles (if `dist[i][i] < 0` after running, a negative cycle exists)
- Graph diameter / eccentricity computation (need distances between all pairs)
- Routing tables computation in small networks

4. How do you identify it's a Floyd-Warshall problem?

Signal phrases (2-minute test): - "shortest path between **every pair** of nodes" / "all-pairs shortest path" - "given a matrix of direct distances, compute the shortest distance between all cities" - "find the city with the smallest number of reachable neighbors within distance threshold" - Graph given as an **adjacency matrix** already (a strong hint — Floyd-Warshall works directly on it) - V (number of nodes) is explicitly small (≤ a few hundred)

5. Can you identify it from constraints?

Constraint clue	Implication
V ≤ ~400–500	$O(V^3)$ fits comfortably within typical time limits
Need distances between **all** pairs, not just from one source	Floyd-Warshall (or V × Dijkstra/Bellman-Ford, but F-W is simpler to code)
Graph given as adjacency matrix	Strong signal — Floyd-Warshall operates naturally on matrices
Edge weights can be negative, but no negative cycle guaranteed	Floyd-Warshall handles negative edges; still must check diagonal for negative cycles
V is large (10^4+)	Floyd-Warshall's $O(V^3)$ becomes infeasible — must use V×Dijkstra or accept single-source only

6. Types / Variants

1. **Classic Floyd-Warshall (shortest distance)** — `dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])` for every intermediate `k`.
2. **Floyd-Warshall for Transitive Closure** — boolean reachability version: `reach[i][j] |= reach[i][k]`

`&& reach[k][j]`.

3. **Floyd-Warshall with Path Reconstruction** — maintain a `next[i][j]` matrix to reconstruct the actual shortest path, not just its length.
4. **Floyd-Warshall for Negative Cycle Detection** — after running, if any `dist[i][i] < 0`, a negative cycle exists.
5. **Min-Max Path variant** — replace `+` / `min` with other operations (e.g., minimize the maximum edge weight on a path, or maximize the minimum edge — "widest path problem").

7. Rationale / Intuition

The algorithm is a beautifully simple dynamic program: **`dist[i][j]` after considering intermediate nodes `{1...k}` = min(not using k, using k as a waypoint)**. Formally: `dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])`. By iterating `k` from 1 to V and updating the entire matrix each time, you progressively allow paths to route through more and more intermediate nodes, until by `k = V` every possible intermediate node has been considered — giving the true shortest path between every pair. The three nested loops (`k`, `i`, `j`) directly encode this DP recurrence, which is why the algorithm is so short to implement (~10 lines) despite being non-obvious to derive from scratch.

Order matters: the `k` loop must be outermost — it represents "the set of allowed intermediate nodes so far," which must be fixed before scanning all `(i, j)` pairs for that generation.

8. Time & Space Complexity

Operation	Time	Space
Floyd-Warshall (all-pairs)	$O(V^3)$	$O(V^2)$
With path reconstruction	$O(V^3)$	$O(V^2)$ extra for `next[][]`
Negative cycle check	O(V) (scan diagonal after run)	—

9. Comparison with Other Patterns

Algorithm	Scope	Negative weights?	Time	Best when
Floyd-Warshall	All-pairs	Yes (no negative cycle)	$O(V^3)$	Small V (≤ ~500), need full distance matrix
Dijkstra (×V runs)	All-pairs (via repetition)	No	$O(V \cdot (V+E) \log V)$	Large sparse V, non-negative weights, all-pairs still needed
Bellman-Ford (×V runs)	All-pairs (via repetition)	Yes	$O(V^2 \cdot E)$	Rare — Floyd-Warshall usually simpler for this case
Bellman-Ford (single)	Single-source	Yes	$O(V \cdot E)$	Just need one source, negative weights
Dijkstra (single)	Single-source	No	$O((V+E) \log V)$	Just need one source, non-negative weights

Decision rule: **need all pairs + V is small → Floyd-Warshall.** Need all pairs + V is large + non-negative weights → run Dijkstra from each node instead (typically faster for sparse graphs). Need just one source → Dijkstra or Bellman-Ford depending on edge signs.

10. Reusable Java Templates

10.1 Classic Floyd-Warshall

```
class FloydWarshall {
    public static final int INF = (int) 1e8;
```

```
    // graph[i][j] = direct edge weight, INF if no direct edge, 0 if i == j
    public int[][] allPairsShortestPath(int[][] graph) {
        int n = graph.length;
        int[][] dist = new int[n][n];
        for (int i = 0; i < n; i++) dist[i] = graph[i].clone();

        for (int k = 0; k < n; k++) {
            for (int i = 0; i < n; i++) {
                if (dist[i][k] == INF) continue; // small optimization
                for (int j = 0; j < n; j++) {
                    if (dist[k][j] == INF) continue;
                    if (dist[i][k] + dist[k][j] < dist[i][j]) {
                        dist[i][j] = dist[i][k] + dist[k][j];
                    }
                }
            }
        }
        return dist;
    }

    // check after running: negative cycle exists if any dist[i][i] < 0
    public boolean hasNegativeCycle(int[][] dist) {
        for (int i = 0; i < dist.length; i++) {
            if (dist[i][i] < 0) return true;
        }
        return false;
    }
}
```

10.2 Floyd-Warshall with Path Reconstruction

```
class FloydWarshallWithPath {
    public static final int INF = (int) 1e9;
    private int[][] dist;
    private int[][] next;

    public void build(int[][] graph) {
        int n = graph.length;
        dist = new int[n][n];
        next = new int[n][n];

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                dist[i][j] = graph[i][j];
                next[i][j] = (graph[i][j] != INF && i != j) ? j : -1;
            }
        }

        for (int k = 0; k < n; k++) {
            for (int i = 0; i < n; i++) {
                for (int j = 0; j < n; j++) {
                    if (dist[i][k] != INF && dist[k][j] != INF
                            && dist[i][k] + dist[k][j] < dist[i][j]) {
                        dist[i][j] = dist[i][k] + dist[k][j];
                        next[i][j] = next[i][k];
                    }
                }
            }
        }
    }

    public java.util.List<Integer> reconstructPath(int u, int v) {
        java.util.List<Integer> path = new java.util.ArrayList<>();
```

```
        if (next[u][v] == -1) return path; // no path
        path.add(u);
        while (u != v) {
            u = next[u][v];
            path.add(u);
        }
        return path;
    }
}
```

10.3 Transitive Closure (Reachability) Variant

```
class TransitiveClosure {
    public boolean[][] reachability(boolean[][] adj) {
        int n = adj.length;
        boolean[][] reach = new boolean[n][n];
        for (int i = 0; i < n; i++) reach[i] = adj[i].clone();

        for (int k = 0; k < n; k++) {
            for (int i = 0; i < n; i++) {
                for (int j = 0; j < n; j++) {
                    reach[i][j] = reach[i][j] || (reach[i][k] && reach[k][j]);
                }
            }
        }
        return reach;
    }
}
```

11. Quick Recognition Checklist (2-minute test)

- ☐ Need shortest paths between **every** pair of nodes, not just one source?
- ☐ Number of nodes is small (roughly ≤ a few hundred)?
- ☐ Graph given as (or convertible to) an adjacency matrix?
- ☐ Possibly need negative-cycle detection or reachability, not just distances?

If 2+ checked → Floyd-Warshall.